

ETL Use case -GCP

This documentation provides a comprehensive overview of the **Crowdfunding ETL Pipeline**, designed using a **Medallion Architecture** on Google Cloud Platform. It transforms raw crowdfunding data into an analytics-ready "Gold" layer.

1. Pipeline Architecture

The pipeline follows a modern cloud-native design, utilizing a "Trigger-Action" pattern.

- **Ingestion (Landing):** CSV files are uploaded to a **Google Cloud Storage (GCS)** bucket.
- **Compute (Cleaning):** A **Cloud Function (Gen 2)** is triggered by GCS events. It uses **Python (Pandas)** to clean junk rows, parse JSON, and standardize headers.
- **Storage (Warehousing):** Data is moved through four stages in **BigQuery**:
 - **Landing:** Raw, untouched data.
 - **Bronze:** Validated data with individual tables cleaned.
 - **Silver:** Unified data; campaign, category, and subcategory tables are joined.
 - **Gold:** Production-ready views for business metrics.

2. Design Decisions

Why Cloud Functions (Gen 2)?

- **Scalability:** Automatically scales based on the number of file uploads.
- **Cost-Effectiveness:** You only pay for the seconds the code actually runs during a file upload.
- **Enhanced Resources:** Gen 2 allows for up to 32GiB of RAM, which was necessary to handle the memory-intensive imports of **pandas** and **numpy**.

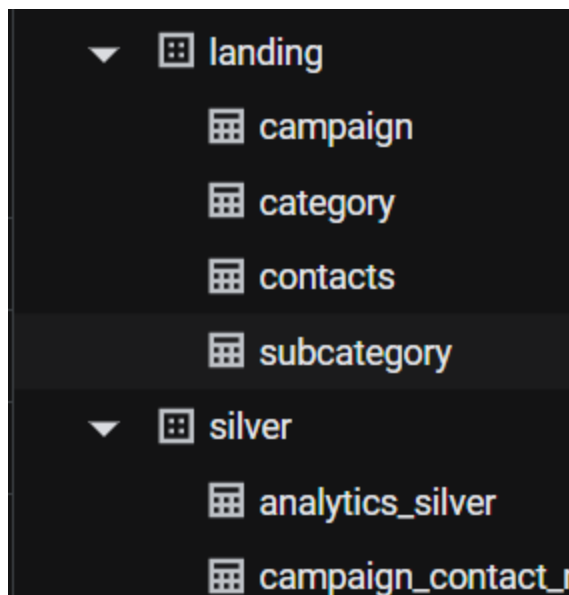
Medallion Data Design

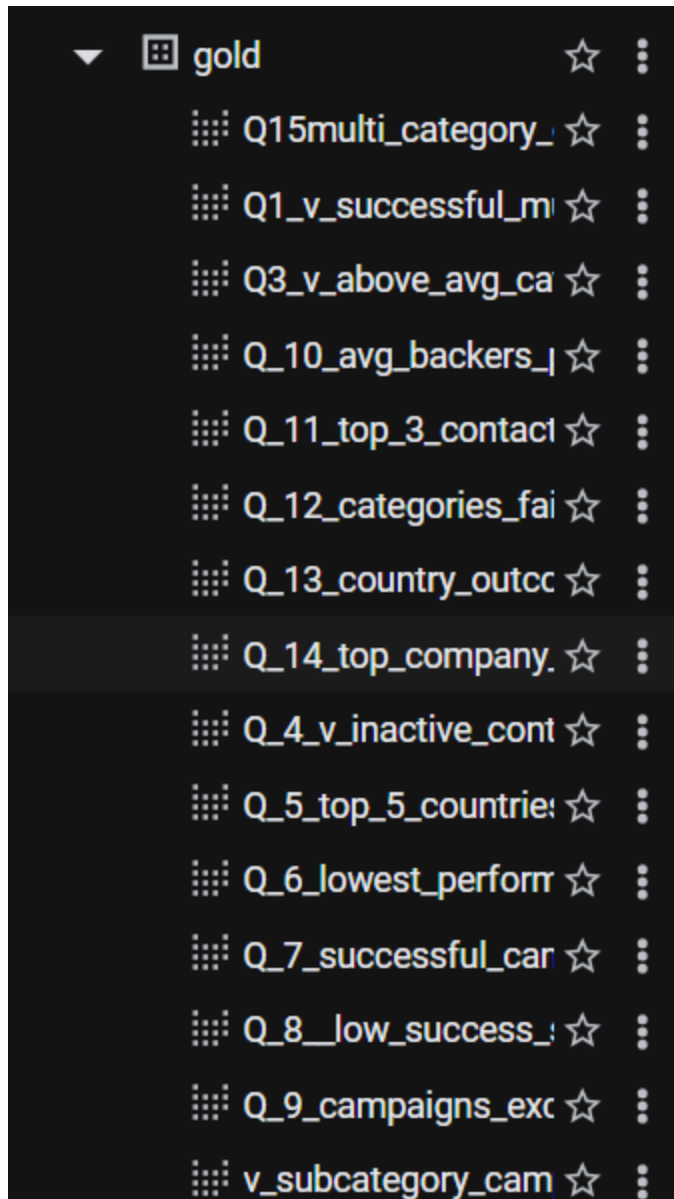
- **Landing vs. Bronze:** We chose to keep a "Landing" dataset to preserve the raw integrity of the data. If the logic fails, we can re-run the pipeline without re-uploading files.
- **Silver Merging:** We use SQL **JOIN** operations in the Silver layer to reduce data redundancy, we have used **COALESCE** to clean columns.

- **Gold layer** :We have exposed gold layer as views to only display needed information to the business teams.

```
🔍 silver table creation 1 ▶ Run 📄 Save query ▾ ⬇ Download 👤 Share ▾ 🕒 Schedule  
1 CREATE OR REPLACE TABLE `etl-use-case-486109.silver.analytics_silver` AS  
2 SELECT  
3   cf_id,  
4   contact_id,  
5   -- Cleaning strings with placeholders  
6   COALESCE(company_name, 'No Company Provided') AS company_name,  
7   COALESCE(description, 'No Description Provided') AS description,  
8   outcome,  
9   COALESCE(country, 'No country Provided') as country,  
10  COALESCE(currency, 'No currency Provided') as currency,  
11  COALESCE(category, 'cat_missing') AS category_id,  
12  COALESCE(subcategory, 'subcat_missing') AS subcategory_id,  
13  
14  -- Imputing missing numbers using averages from the Python-generated table  
15  COALESCE(goal, (SELECT AVG(goal) FROM `etl-use-case-486109.landing.final_analytics`)) AS goal,  
16  COALESCE(pledged, (SELECT AVG(pledged) FROM `etl-use-case-486109.landing.final_analytics`)) AS pledged,  
17  COALESCE(backers_count, CAST((SELECT AVG(backers_count) FROM `etl-use-case-486109.landing.final_analytics`) AS  
18  INT64)) AS backers_count,  
19  
20  -- Preserving original dates  
21  launch_date as launched_date,  
22  end_date  
23 FROM `etl-use-case-486109.landing.final_analytics`;
```

Architecture





Performance Optimizations

- **Internal Initialization:** Database clients ([bigquery.Client](#)) are initialized **inside** the function entry point. This avoids "Cold Start" timeouts and ensures the container passes Google's port 8080 health check.

3. Implementation Details

Data Cleaning Logic (Python)

The pipeline handles two distinct data structures within a single function:

- **Contacts:** Skips the first 3 lines of non-data notes and uses a `json.loads` loop to extract nested contact information.
- **Campaigns:** Converts Unix timestamps (integers) to standard SQL `DATE` format using:
`$$\text{date} = \text{pd.to\_datetime}(\text{unix\_val}, \text{unit='s'})$$`

BigQuery Layer Logic (SQL)

To ensure the **Gold** layer is dashboard-ready, we implemented **Null-Coalescing** and **Averaging**:

- **String Nulls:** `COALESCE(company_name, 'No Company Provided')`
- **Metric Imputation:** Missing goal or pledge amounts are replaced with the global average of the dataset to prevent skewed analytics.

4. Maintenance & Error Handling

- **Logs:** All execution details are sent to **Cloud Logging**. Deployment failures (like the Port 8080 error) are monitored via the Cloud Run revision history.
- **Schema Evolution:** The pipeline uses `df.columns` normalization to ensure that minor changes in CSV headers (e.g., "Currency" vs "currency") do not break the BigQuery load.

Severity	Timestamp	Summary
> *	2026-02-02 19:05:05.201 IST	Processing file: crowdfunding.csv
> *	2026-02-02 19:05:05.251 IST	Processing file: contacts.csv
> i	2026-02-02 19:05:05.304 IST	Default STARTUP TCP probe succeeded after 1 attempt for container "worker" on port 8080.
> *	2026-02-02 19:05:05.442 IST	/workspace/main.py:67: FutureWarning: The behavior of 'to_datetime' with 'unit' when parsing strings is deprecated. I...
> *	2026-02-02 19:05:05.451 IST	Processing file: subcategory.csv
> *	2026-02-02 19:05:05.464 IST	/workspace/main.py:68: FutureWarning: The behavior of 'to_datetime' with 'unit' when parsing strings is deprecated. I...
> *	2026-02-02 19:05:05.694 IST	Successfully loaded 9 rows into category
> *	2026-02-02 19:05:05.705 IST	/layers/google.python.pip/pip/lib/python3.10/site-packages/google/cloud/bigquery/_pandas_helpers.py:484: FutureWarnin...
> *	2026-02-02 19:05:05.731 IST	/layers/google.python.pip/pip/lib/python3.10/site-packages/google/cloud/bigquery/_pandas_helpers.py:484: FutureWarnin...
> *	2026-02-02 19:05:05.763 IST	Exception on / [POST] Traceback (most recent call last): File "/layers/google.python.pip/pip/lib/python3.10/site-pack...
> *	2026-02-02 19:05:05.868 IST	/layers/google.python.pip/pip/lib/python3.10/site-packages/google/cloud/bigquery/_pandas_helpers.py:484: FutureWarnin...
> *	2026-02-02 19:05:09.508 IST	Successfully loaded 1000 rows into contacts
> *	2026-02-02 19:05:10.323 IST	Successfully loaded 24 rows into subcategory

5. Data Cleaning & Transformation Summary

The pipeline performs three levels of transformation: **Structural Normalization**, **Content Cleaning**, and **Enrichment**.

A. Structural Normalization

Before the data touches BigQuery, the Python script ensures the physical structure is consistent regardless of the source file's formatting.

- **Case Normalization:** All headers are converted to lowercase.
- **Symbol Replacement:** Spaces and hyphens in column names are replaced with underscores (e.g., **Category & Sub-Category** becomes **category_&_sub_category**).
- **Whitespace Stripping:** All string values and headers are stripped of leading/trailing whitespace to prevent "invisible" duplicates.

B. Specialized Transformation Logic

The script detects the file type and applies specific logic to handle data quirks:

1. The Contacts Parser

The **contacts.csv** contained non-standard formatting that would crash a normal loader.

- **Header Offset:** The script ignores the first 3 rows of the file, which contained metadata/notes rather than data.
- **JSON Extraction:** The core data was stored as a stringified JSON object in a single column. The script uses a **try-except** block with **json.loads()** to unpack these into distinct **contact_id**, **name**, and **email** columns.

- **String Splitting:** The `name` column is split at the first space to create `first_name` and `last_name` fields.

2. The Campaign Date Engine

Raw crowdfunding data uses **Unix Timestamps** (seconds since Jan 1, 1970). These are unreadable for humans and most BI tools.

- **Conversion:** We applied the formula:
`$$\text{Date} = \text{pd.to\_datetime}(\text{Timestamp}, \text{unit='s'}).dt.date$$`
- **Impact:** This transformed integers like `1608292800` into readable SQL DATE objects like `2020-12-18`.

C. Category & Subcategory Mapping

To support hierarchical reporting, the pipeline breaks down combined strings:

- **The Split:** The logic looks for the `/` delimiter in the category column.
- **The Result:** A value like `food/food trucks` is split into two distinct columns: `category` ("food") and `subcategory` ("food trucks").

D. The "Silver" Layer Imputation (Data Quality)

In the silver stage, we perform "Data Imputation" to ensure dashboards never show broken or empty components:

- **Categorical Fill:** Replaces `NULL` values in string columns with standardized placeholders (e.g., "No Company Provided").
- **Numerical Imputation:** Uses **Statistical Mean Imputation**.
 - *Logic:* If a `goal` or `pledged` amount is missing, the SQL calculates the average of all other campaigns and inserts that value.
 - *Reasoning:* This prevents `$0$` or `NULL` values from dragging down the accuracy of global financial aggregates.